Multicamera video editing with self-attention and temporal and camera embeddings (English AI translation)

Universitat Oberta de Catalunya

Òscar Casals Morro

This AI translation has only been reviewed by the author, Òscar Casals Morro. Below you can find the advisor and instructor that oversaw the catalan version of the thesis.

Program name
Area 4

Final Master Thesis Advisor
Ismael Benito Altamirano
Instructor in charge of the course
David Masip

Delivery Date
05/2026

Final Project Sheet

Title:	Multicamera video editing with self-attention and temporal and camera embeddings
Author's name:	Òscar Casals Morro
Thesis Advisor's Name:	Ismael Benito Altamirano
Name of the PRA:	David Masip
Delivery Date:	05/2026
Degree:	Master in Data Science
Final Thesis Area:	Area 4
Language:	English
Keywords	Multicamera, self-attention, frame.

Abstract in catalan

Quan es treballa en sistemes multicàmera és molt important seleccionar el punt de vista correcte a mostrar en cada fotograma, amb l'objectiu d'agilitzar aquest procés, en aquest treball es crearà un model capaç de suggerir quina càmera utilitzar en cada moment del vídeo.

Per a crear aquest model, es replicarà el model descrit a l'article "*Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*" amb dos *backbones*, i es determinarà quin dels dos és el més indicat.

Als resultats s'espera obtenir un model que pugui seleccionar els fotogrames a utilitzar en cada moment.

Abstract in english

When working with multicamera systems, it is very important to select the correct viewpoint to display in each frame. To streamline this process, this work will create a model capable of suggesting which camera to use at every moment of the video.

To create this model, the architecture described in the article “I*mproving Multi-Camera View Recommendation with Temporal and Camera Embedding*” will be replicated with two backbones, and at the end of the project the most suited for the task will be determined.

Results are expected to yield a model that can select the frames to use at each moment as precisely as the model described in the aforementioned article.

Index

1. Introduction **1**
1.1. Context and justification of the project 1
1.2. Objectives of the Project 3
1.3. Impact on sustainability, ethics, and diversity 3
1.4. Approach and method followed 5
1.5. Planning 6
1.6. Brief summary of the products obtained 7
1.7. Brief description of the other chapters of the thesis 8
1.8. Declaration on the Use of Generative Artificial Intelligence 8
2. State of the art **8**
3. Materials and methods **15**
4. Results **24**
5. Conclusions and future directions **33**
6. Glossary **38**
7. Bibliography **39**

1. Introduction

1.1. Context and justification of the project

A multi-camera system is a filming technique in which several cameras simultaneously record the same scene or set from different viewpoints. Afterwards, during the editing process, the best shots from each camera are selected and combined into a single final video.

These systems are essential for broadcasting concerts, television shows, and sporting events, where the time between the live recording and its broadcast or commercial release is often very short.

In addition, this filming methodology is widely used in movies and television series, as it allows editors to capture every possible angle and select the most appropriate one during post-production, reducing the need to re-record scenes if a particular shot is unsatisfactory.

Despite its advantages, multi-camera production presents a major drawback: the editing process is highly labor-intensive.

Since selecting the most appropriate viewpoint when combining footage from multiple cameras is crucial to the viewer's experience, a large number of professional editors who spend many hours reviewing recordings from every camera are required. Their decisions rely not only on the content of each candidate frame but also on established cinematographic principles, such as progressively transitioning from a wide shot to a closer shot when introducing a new scene.

Until recently, no solution had been sufficiently robust to automate this process while preserving the benefits of professional multi-camera production, making its adoption in real-world environments impractical.

However, considering the rapid advances in Deep Learning over the last few years, an important question arises: “Can Deep Learning speed up this process”?

This question is addressed in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[1], where the authors introduce a dataset composed of unedited multi-camera recordings from four different domains: concerts, sporting events, gala shows, and television game shows; and use it to train a model capable of selecting the most appropriate frame at each moment in a multi-camera environment.

The dataset introduced in that work has been used to train several models that seek to improve upon the one presented in that paper. For example, *Pseudo Dataset Generation for Out-of-Domain Multi-Camera View Recommendation*[2] expands the original dataset by generating synthetic multi-camera environments through clustering, while *Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3] proposes a more computationally efficient architecture than the original approach.

The objective of this project is to use the same dataset to develop a model capable of recommending the most appropriate viewpoint in a multi-camera environment, with the aim of reducing the workload of professional video editors.

To achieve this, the architecture proposed in *Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3] was reproduced using two different feature extraction backbones: SwinV2 Tiny (swinv2_tiny_window8_256)[4] and MobileNetV3 Small (mobilenetv3_small_100.lamb_in1k)[5]. Comparing these two alternatives makes it possible to evaluate the extent to which the choice of backbone affects the model's overall performance.

From a personal perspective, this project also provided an opportunity to apply the knowledge acquired during the Deep Learning course of the Master's Degree in Data Science to a real-world computer vision problem.

1.2. Objectives of the Project

- Develop a model capable of deciding the best frame for a certain moment in a multicamera video.

- Compare the final model with the metrics reported in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[1] to check if there has been an improvement like in "*Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*"[3].

- Train the model with two different backbones, and determine if there is an impact on the results.

- Evaluate the quality of the final model.

1.3. Impact on sustainability, ethics, and diversity

Sustainability:

As discussed in *The green paradox: The climate, environmental, and sustainability implications of artificial intelligence*[6], the rapid advancement of artificial intelligence has had a significant environmental impact. This is mainly due to the following factors:

- The continuous expansion of data centers and increasingly large AI models has led to a growing demand for essential resources such as electricity, water, and raw materials.

- The construction and operation of these data centers has also contributed to increased CO_2 emissions.

However, these environmental costs are primarily associated with large-scale models trained on massive datasets that depend on extensive computing infrastructures to function.

The model developed in this project has considerably lower computational requirements, as it can be trained and executed on a reasonably powerful personal computer rather than requiring access to large computing clusters.

Furthermore, a lightweight architecture was deliberately selected, further reducing the environmental impact associated with both training and inference.

Ethics and Social Responsibility:

From a social perspective, the proposed model is not intended to replace professional video editors but rather to assist them by reducing the time required to edit multi-camera productions. The model only recommends the most appropriate viewpoint when a camera switch is to be made.

Creative decisions, such as determining the optimal moment to switch cameras or assessing whether the model's recommendation is artistically appropriate, remain entirely under the control of a professional editor.

If successfully adopted, this approach could improve editing workflows in multi-camera productions by allowing editors to focus on higher-level creative decisions instead of manually reviewing every available camera angle.

From an ethical perspective, the proposed model is neutral. It does not promote or discourage any particular behavior; it simply recommends the viewpoint that best matches the patterns learned during training. Consequently, the ethical implications of the generated content ultimately depend on how the product is used by its users.

Diversity, Gender and Human Rights:

Since the objective of the model is to recommend the most appropriate camera viewpoint within a multi-camera production, its use inherently requires the ability to distinguish between different camera shots. Consequently, the system is not directly accessible to users with severe visual impairments. This limitation arises from the nature of the task itself and is therefore outside the scope of this project.

Regarding fairness with respect to gender, ethnicity, or other demographic characteristics, none of the studies using the TVMCE dataset reviewed during this project reported evidence of systematic bias affecting any particular group. Therefore, there is no indication that the proposed model should exhibit discriminatory behavior toward any demographic group.

Finally, this work does not infringe upon intellectual property rights, as all external resources and prior work are properly cited throughout the document. The developed system is intended solely to support professional video editors in multi-camera environments. Consequently, it remains the responsibility of the end user to ensure that any videos produced using the model comply with all applicable laws and copyright regulations

1.4. Approach and method followed

For this project, the model described in "*Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*"[3] will be replicated and compared with "*Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*"[1]

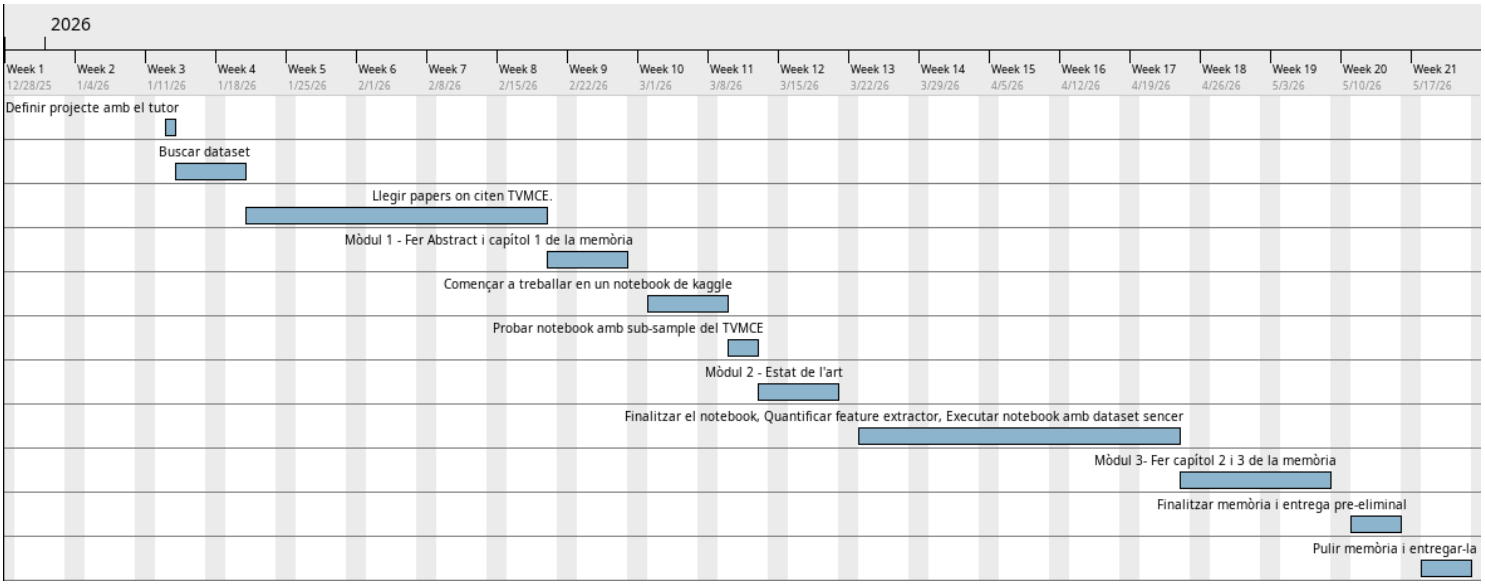
The reasons why this architecture was chosen were:

- It presents a relatively short training time in comparison with other models.
- Its accuracy is superior to previous models
- Since its training does not require a large amount of resources, the model can be developed in an environment that is not at the level of clusters

For feature extraction, two backbones will be tested: *swinv2_tiny_window8_256*[4] and a less powerful one called *mobilenetv3_small_100.lamb_in1k*[5].

The reason behind this comparison is to determine whether using a less powerful backbone impacts the results or, otherwise, could reduce the amount of resources required by the model.

1.5. Planning



Name	Begining date	Final date	Duration (days)
Define the project with the advisor	1/13/26	1/13/26	1
Look for a dataset	1/14/26	1/20/26	5
Read papers that cite TVMCE.	1/21/26	2/19/26	22
Module 1 - Abstract and chapter 1 of the thesis.	2/20/26	2/27/26	6
Getting started with a Kaggle notebook	3/2/26	3/9/26	6
Test notebook with TVMCE subsample	3/10/26	3/12/26	3
Module 2 - State of the Art	3/13/26	3/20/26	6
Finish the notebook, run the notebook with the entire dataset. If there's time, quantify the feature extractor.	3/23/26	4/23/26	24
Module 3 - Write Chapters 2 and 3 of the thesis	4/24/26	5/8/26	11
Finalize the report and make a preliminary delivery.	5/11/26	5/15/26	5
Polish the memory and deliver it.	5/18/26	5/22/26	5

1.6. Brief summary of the products obtained

This project presents a model capable of recommending the most appropriate viewpoint to display at each moment in a multi-camera environment.

Since there is currently no publicly available implementation of a model capable of addressing this problem, another objective of this project is to provide an open implementation that can serve as a foundation for future research and encourage further innovation in the field of automatic multi-camera video editing.

The proposed model is based on the architecture presented in *Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3].

If the project objectives are successfully achieved, the time required to produce a single edited video from multiple camera viewpoints could be significantly reduced. Instead of requiring a team of professional editors to spend hours reviewing footage from several cameras to determine the most appropriate shot at each moment, the proposed model automatically generates viewpoint recommendations. Editors would then only need to review alternative camera angles when they disagree with the model's selection.

Furthermore, this project provides a public implementation of the model proposed in *Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3]. This implementation can serve both as a baseline for developing new approaches and as a starting point for future research aimed at advancing the state of the art in automatic multi-camera video editing.

1.7. Brief description of the other chapters of the thesis

- **Chapter 2:** State of the art, in other words, previous projects that tackled the same issue.
- **Chapter 3:** The inner workings of the model will be described, and several design decisions will be justified.
- **Chapter 4:** The quality of the model will be evaluated through some metrics.
- **Chapter 5:** Conclusions of the project, a critical look at its planning, and an exploration of future lines of work.
- **Chapter 6:** Definition of the most relevant terms and acronyms used throughout the thesis.
- **Chapter 7:** Bibliography.

1.8. Declaration on the Use of Generative Artificial Intelligence

For this project, Gemini was used to produce Figure 5.

To do it, a description of how the feature extractor of the model works was provided to the AI, and then a prompt asking if it could create a schema from it was made.

The resulting image was edited to hide an excess of information.

Finally, exclusively for this English version of the thesis, ChatGPT was used to translate the thesis from Catalan to English in order to speed up the translation process.

2. State of the art

Multi-camera systems are essential for the production of most audiovisual content today. This technique, which involves recording the same scene from multiple viewpoints and then using the best angles to produce a single video, is widely used in sports, galas, competitions, concerts…

However, this methodology presents a major problem: the time required to produce the final video.

Creating a video from multiple sources requires a professional to watch each frame captured by the different cameras for hours and select the best one for every moment. This choice is not based solely on the content of each candidate, but also on the frames already selected for the final video and on cinematic principles such as the following, described in the book *The Technique of Film Editing*[7]:

- Do not show the same viewpoint for a long time.
- Introduce a new scene from a general point of view and gradually move towards one closer to the main action.

Figure 1 shows an example of the editing process in multi-camera environments generated with the TVMCE dataset.

In the example, you can see how the video moves from a general viewpoint to a closer one, as described in the earlier cinematic principles.

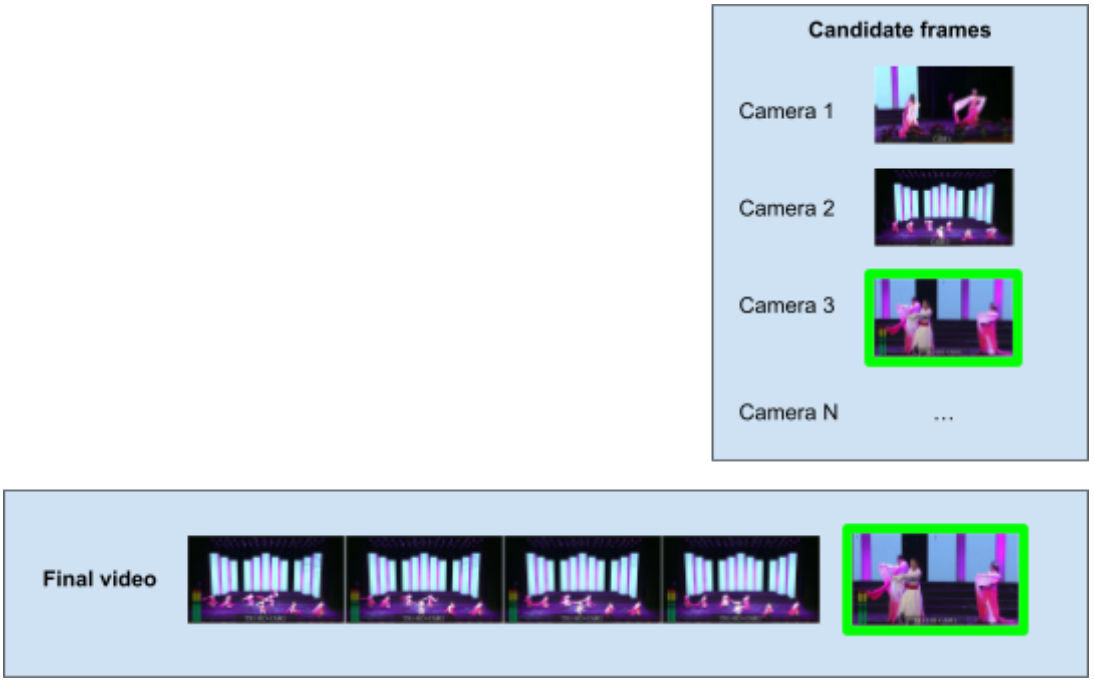


Figure 1: *Representation of the editing process in a multicàmera environment taken from the TVMCE dataset*[1].

With recent advances in machine learning, specifically in Deep Learning, several models have been developed to streamline this process.

However, when it comes to developing them, there is a major problem: the lack of public datasets with recordings from different cameras and the final video with the selected frames. This makes it very difficult to train a model.

Some papers like *Real Time GAZED: Online Shot Selection and Editing of Virtual Cameras From Wide-Angle Monocular vídeo Recordings*[8] avoid the problem entirely by simulating multiple cameras from a wide shot, dividing the scene into windows and choosing the best one for each moment.

This approach, however, is intended for productions where only a single wide shot is available and therefore does not account for scenarios with more than one camera positioned around the scene.

Another possible solution is the one proposed in the paper *Pseudo Dataset Generation for Out-of-domain Multi-Camera View Recommendation*[2], where the size of existing multi-camera datasets is increased through already edited linear videos.

The methodology used in this paper is based on two facts:

- Many cuts in a video are the result of camera changes.
- In a multi-camera environment, the cameras are stationary and responsible for recording perspectives at different scales (from wide shots to those closer to the action).

Thanks to these two points, it is possible to perform clustering on the different frames that make up a video, assigning to each member of a group the same label, which corresponds to the “camera” they represent. From this point onward, each of these simulated cameras will be called a “pseudo-camera”.

Once the pseudo-multicamera environment has been created, it is necessary to define the various candidates from among which, in a real multicamera setup, one would be chosen to produce the final video. This process is carried out as follows:

1. The features of each image are extracted using the ResNet50[9] neural network pre-trained on ImageNet[10].

2. Cosine similarity is computed between the frames belonging to different pseudo-cameras. The most similar frames across different pseudo-camera groups are selected as candidates, while the frame used at the corresponding timestamp in the original video is treated as the ground-truth selection that the model is trained to predict.

This simulation aims to improve the coverage of existing datasets and thus enhance model generalization.

Another attempt to simulate multi-camera environments is the one in *ReCamMaster: Camera-Controlled Generative Rendering from A Single video*[11], where, although they are not aiming to develop a model to select the best frame in a multi-camera environment, they also encounter the problem of a lack of datasets.

In this paper, the way to solve the problem is to simulate multi-camera environments using the famous graphics engine *Unreal Engine 5*[12].

The most comprehensive dataset currently available, and the one used by most of the studies reviewed in this work, is the TVMCE dataset introduced in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[1].

The paper introduces a dataset consisting of unedited multi-camera recordings from four different domains: concerts, sporting events, gala shows, and television game shows. Across these four domains, the dataset contains 88 hours of unedited footage captured from multiple cameras, which was used to produce 14 hours of broadcast-ready edited videos.

From this dataset, several models tackling the same issue as in this project have been developed, most of them treating it as a classification problem.

The paper *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[1] proposes an architecture based on two transformers: a temporal one that processes previous frames to those under consideration, and a contextual one that handles the frames being considered for a given moment.

The proposed model, as can be seen in Figure 2, is composed of two components:

- **TC-Transformer:**
  - The features from each frame are extracted and kept in a numeric vector (*feature vector*)
  - Two transformers are used:
    - A temporal one, that models historical information using previous frames to those in consideration.
    - A contextual one, that models the frames considered for a certain moment, in other words, the candidate frames.
  - For feature extraction two backbones were tested: ViVit[13] and video-Swin[14], both pre-trained with *Kinetics-400*[15], being vídeo-Swin[14] the one that presented better results.
- **Multi-layer perception (MLP):**
  - The results of each transformer are introduced as input to an MLP, which assigns a score on how adequate each frame is.

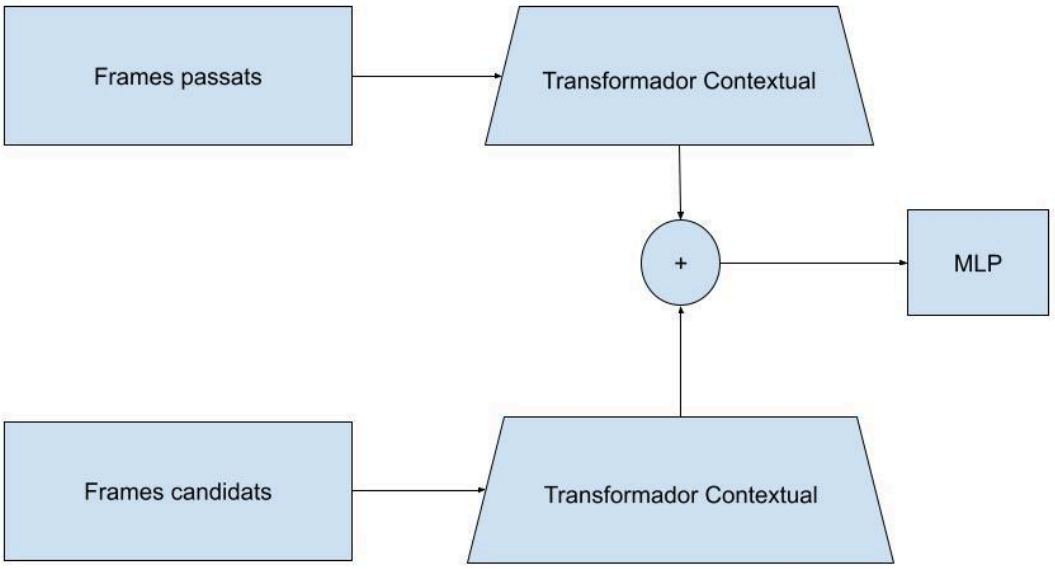


Figure 2: Schema of how the model presented in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[1] works.

The aforementioned model, though, presents issues outside the domains it has been trained with. To improve generalisation, the paper *Pseudo Dataset Generation for Out-of-domain Multi-Camera View Recommendation*[2] artificially increases the size of

the TVMCE dataset via synthetic multicamera environments created using pseudo-cameras.

This approximation still does not manage to improve generalisation, although it does improve precision by 68% for the domains included in TVMCE.

The methodology of using synthetic multicamera environments to improve the TVMCE dataset has been used in other projects that focus on a specific field, like in *When and How to Cut Classical Concerts? A Multimodal Automated vídeo Editing Approach*[16], where they focus on classical music concerts.

On the other hand, the paper *Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3] seeks to improve the previously described models incorporating two cinematografic principles:

1. Do not show the same viewpoint for a long time.
2. Progress from a general viewpoint to a narrower one.

For the model to learn these principles, the temporal distance between a frame and the moment when a camera switch happens, as well as the camera ID, have been added to the feature vector as temporal and contextual embeddings.

Unlike the TC-Transformer, this architecture is also characterized by not relying on separate temporal and contextual transformers. Instead, it models the relationships between past frames and candidate frames using *self-attention*[17].

As seen in figure 3, the architecture consists of three components:

- **Feature extraction:**
  - The features from each frame are extracted and kept in a numeric vector (feature vector)
  - The following information is added to the feature vector:
    - Number of frames between a past frame and the candidates (frame offset) as a *positional embedding.*
    - The camera ID as another embedding.
  - The backbone used is *Swin Transformer*[18] pre-trained with *ImageNet*[10].
  - The feature extractor is applied to all frames, both past and candidates.

- **Past-candidate mixer (Self-attention layer):**
  - All feature vectors are concatenated into a single sequence.
  - Context is added to the vectors through *self-attention*.

- **Multi-layer perception (MLP):**
  - The feature vectors with context corresponding to the candidate frames are given as input to an MLP with the following architecture:
    - A normalisation layer.
    - A linear layer.
    - GeLU activation function.
    - A final linear layer.

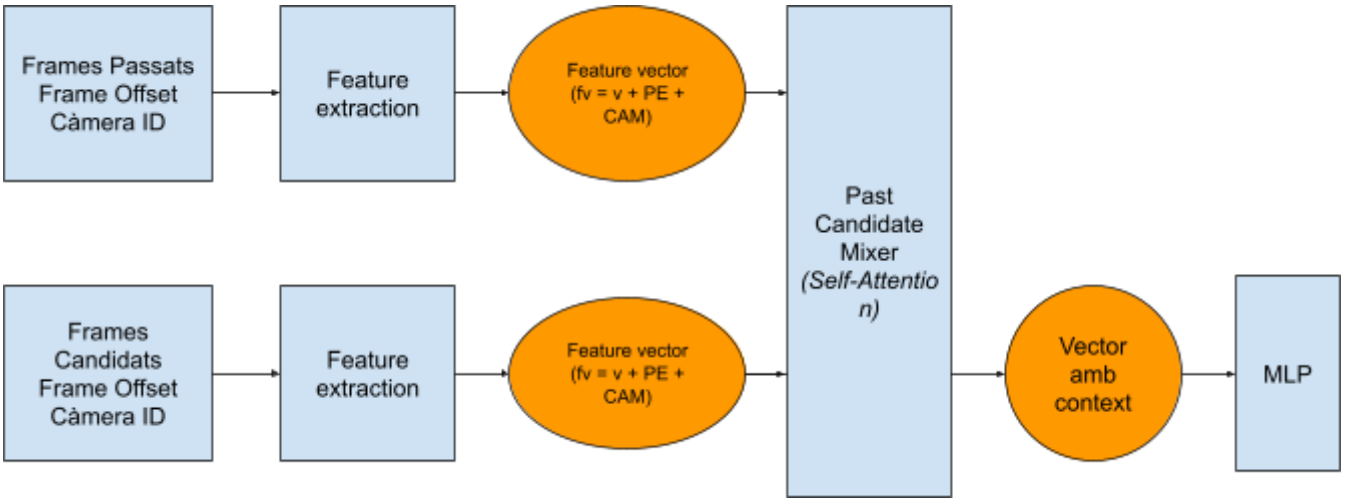


Figure 3: Graphic representation of the architecture described at I*mproving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3]

This framework possesses a precision 14% superior to the model described in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[1], and training time is reduced by 90%.

3. Materials and methods

The TVMCE dataset was used for both training and evaluating the model. It consists of six TAR archives, each containing the frames captured by the different cameras for a specific video, along with the frames selected for the final edited version.

The TVMCE dataset possesses footage of the following domains: concerts, sporting events, gala shows, and television game shows; this makes it a good fit for the project as multi-camera environments are commonplace in these fields. Figure 4 shows the proportion of frames that belong to each category.

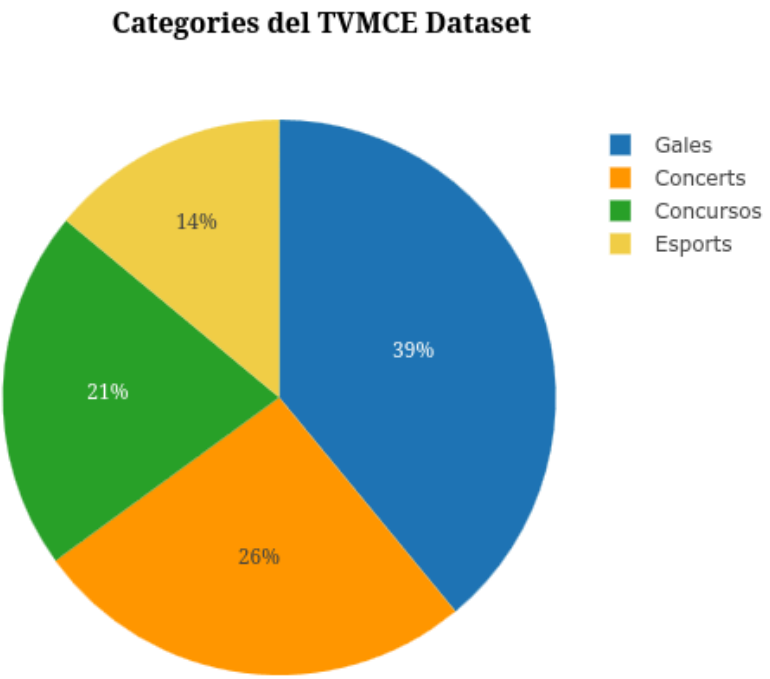


Figure 4: *Categories of TVMCE dataset*[19]*. Graph made with Pie Chart Creator*[20].

In addition to the video footage, the dataset also includes two JSON files. One contains the 7265 samples used to train the model proposed in *Temporal and Contextual Transformer for Multi-Camera Editing of TV*

Shows[19] (hereafter referred to as the TC-Transformer for convenience), while the other contains the 2058 samples used for evaluation.

Each sample in the JSON files contains the following information:

- The frames preceding a given timestamp, together with the ID of the camera used for each frame (hereafter referred to as past frames).
- Whether a camera switch was considered at that timestamp.
- The candidate frames available for selection, together with the ID of the camera from which each candidate originates

Using this information, the temporal distance between each past frame and its corresponding candidate frames (referred to as the frame offset) was computed. Subsequently, as *Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3] shows that past frames located too far from the candidate frames have little influence on the model's predictions, frames more than 500 frames away from the candidate under consideration were discarded. This reduces both computational cost and the amount of noise introduced during training.

Furthermore, since the objective of this project is to develop a model capable of selecting the most appropriate viewpoint to switch to, rather than determining when a camera switch should occur, the training and test sets were filtered to retain only those samples in which a camera switch takes place.

After this filtering process, the training set contained 4034 samples, while the test set contained 1087 samples.

To assess whether the model exhibited signs of overfitting during training, the training set was further divided into two subsets. The first subset, containing 90% of the samples, was used to train the model, while the remaining 10% was used as the validation set.

In this work, the TVMCE dataset was not extended through clustering. Although this technique can be used to generate pseudo-multi-camera videos, it also removes part of the contextual information associated with each frame, which may negatively affect the reliability of the evaluation results. Therefore, only the data available in the original TVMCE dataset were considered.

Similarly, the dataset introduced in *ReCamMaster: Camera-Controlled Generative Rendering from A Single vídeo*[11] was not used. The objective of this project was to train and evaluate the proposed model using footage from real multi-camera productions, allowing its performance to be validated under realistic conditions.

For training and evaluating the model GPU NVIDIA GeForce RTX 3060 Laptop was used. The code was written in Python using the library PyTorch[21].

Regarding data preparation, the initial approach was to use Python's standard *tarfile* library to access the compressed TAR archives directly, avoiding the need to extract them beforehand and thereby reducing the storage requirements needed to run the project.

However, this approach significantly increased the data loading time and consequently slowed down the training process, creating a bottleneck. For this reason, the files were ultimately extracted before training the model.

The architecture used is the same as in *Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3], due to its lower training time and higher precision than the model presented in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Show*[19].

This architecture is characterized by incorporating the frame offset and camera ID into the feature vector extracted from each frame, enabling the model to learn the following cinematographic principles:

- Do not show the same camera for a long time.

- Introduce a scene with a wide shot and gradually progress to a narrower viewpoint.

Furthermore, by incorporating the frame offset and camera ID into each frame's feature vector, the architecture eliminates the need for the temporal and contextual transformers that make up the TC-Transformer. Instead, a self-attention mechanism is used to capture both temporal and contextual information.

This reduces both training time and the amount of resources required for each inference.

Before constructing the model, a custom *Dataset* class was implemented to preprocess the JSON files and retrieve the data required by the feature extractor.

Specifically, the pre-processing this class performs is the following:

- ➢ Removing all samples in which no camera switch was considered.
- ➢ Computing the frame offset for each past frame and discarding samples in which it exceeds 500.
- ➢ When a sample is requested, the class gathers the following information and returns it as tensors, the variables Pytorch[21] works with:
    - ○ The candidate frames, which are resized to 256 × 256 to match the input resolution expected by the pretrained backbone.
    - ○ The camera ID associated with each candidate frame.
    - ○ The past frames preceding the candidate frames.
    - ○ The frame offset between each past frame and the candidate frames.

- The ID of the camera being used immediately before the camera switch.
- The ID of the camera corresponding to the shot ultimately selected in the final edited video.

As seen in figure 6, the model is divided in three components:

- **Feature extractor:**

  The feature extractor is the first component executed within the architecture and is applied independently to both the candidate and past frames.

  It receives as input the information provided by the custom *Dataset* class: the frames, the camera IDs, and the frame offset. Its output is a feature vector that incorporates both the frame offset and the camera ID. To construct this feature vector, the following steps are performed:

  - Visual features are extracted from each frame and encoded into a feature vector (v) using the transformer *swinv2_tiny_window8_256*[4], specifically, the *ImageNet-1k*[22] pre-trained version available in *timm*[23] was used.

  - The frame offset is incorporated into the feature vector as a positional embedding (PE).

  - The camera ID is incorporated into the feature vector as a learnable embedding (CAM).

  As a result, the final feature vector (fv) can be expressed as: $fv = v + PE + CAM$.

  The main advantage of this approach is that, since *swinv2_tiny_window8_256*[4] is already pretrained on the *ImageNet-1k*[22] dataset, only the camera embedding needs to be learned during training.

Figure 5 illustrates the overall structure of the feature extractor.

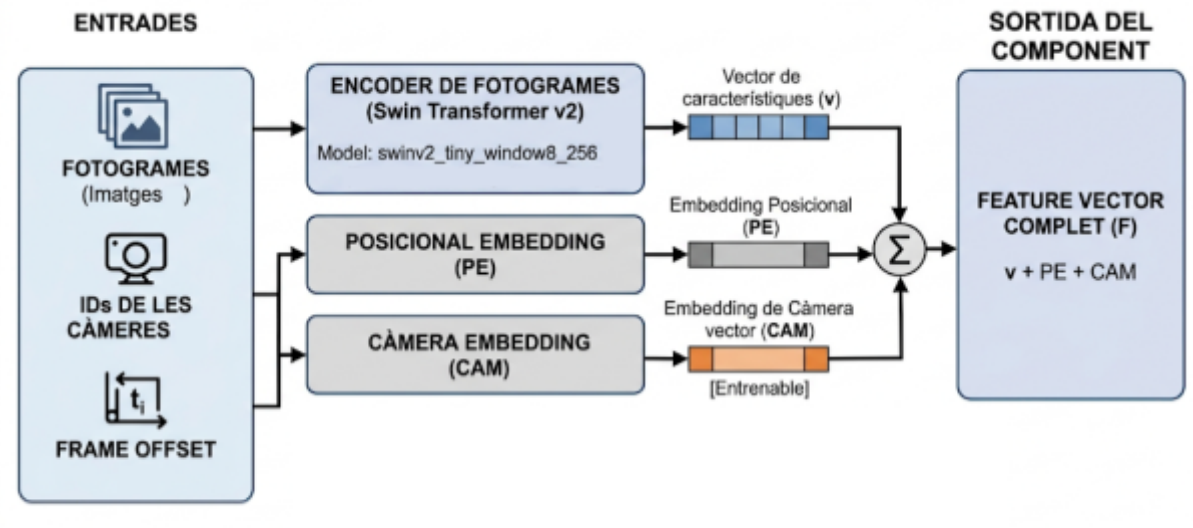


Figure 5: *Schema of the feature extractor made with Gemini*

- **Past-candidate mixer (*Self-Attention* layer):**

  Once all *feature vectors* have been generated, all of them are concatenated into a sequence so *self-attention*[17] can be used.

  By apliying *self-attention*[17] each feature vector gains context on the different candidates and past frames.

- **Multi Layer Perceptron (MLP):**

  Once the feature vectors with context have been obtained, those corresponding to the candidate frames are extracted and passed through an MLP, which assigns a score to each candidate.

  The more suitable a candidate frame is, the higher the score it receives. The candidate with the highest score is then selected as the viewpoint to be used in the final edited video.

The MLP is composed of the following layers:

- A normalization layer applied to the *input*.
- A linear layer.
- GeLU activation function.
- A final linear layer.

As it can be seen in figure 6, the only components that require training are the *camera embedding* from the feature extractor, the past-candidate mixer, and the MLP.

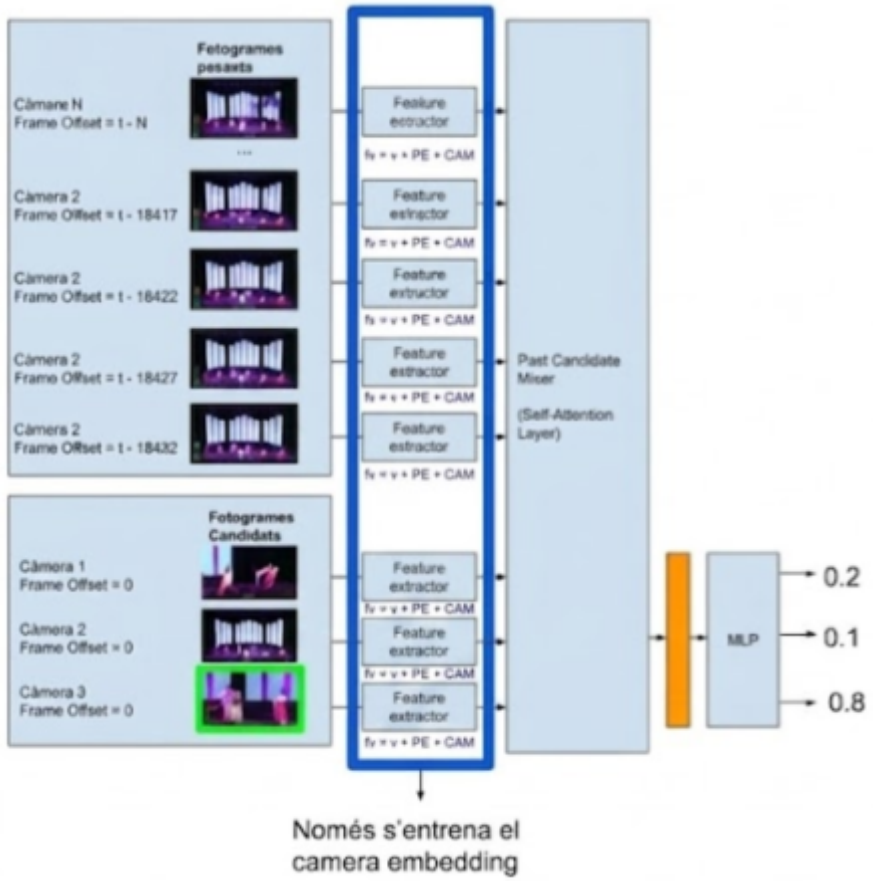


Figure 6: *Model architecture, the parts that do not require training are highlighted in dark blue.*

The model was trained using the Binary Cross-Entropy (BCE) loss function, implemented through PyTorch's *BCEWithLogitsLoss*.

Although the task consists of selecting a single camera from a set of candidate viewpoints, the model outputs an independent score for each candidate. Consequently, the problem can be formulated as a binary classification task in which each candidate has only two possible labels: selected (1) or not selected (0).

This formulation makes it possible to use the BCE loss function, which compares the scores predicted by the model with the binary labels assigned to each candidate camera.

The model was optimized using AdamW[24], a widely adopted variant of the Adam optimizer commonly used in deep learning.

Finally, the model was trained using a learning rate of 1e-5, 10 epochs, and a batch size of 4. This configuration allowed the model to be trained in a relatively short amount of time while achieving good performance.

The reason why a batch size of 4 was chosen instead of 2, like in *Improving Multi-Camera View Recommendation with Temporal and Camera Embeddings*[3], was that, since the project was conducted under limited computational resources, it was considered important to evaluate whether a slightly larger batch size could reduce the training time while maintaining good model performance.

To measure the effectiveness of the training and the model, the following metrics have been used:

- Loss: The difference between predicted and real values. The closer to 0, the more similar the results are to the real values.

- Accuracy: Measures the proportion of correct predictions relative to the total. It is calculated as follows:

$$\frac{\text{Correct predictions}}{\text{Number of predictions}}$$

- Precision: Measures the reliability of the model's predictions

  Considering that, in this case, the true positives are the instances in which a camera has been selected correctly, and the false positives are the times when it has been chosen erroneously.

  Precision can be computed as follows:

$$\frac{\text{True positives}}{\text{True positives + False positives}}$$

- Recall: Measures the model's ability to identify the correct camera.

  Considering that in this scenario false positives correspond to all the times a camera has been wrongly discarded, it is computed as follows:

$$\frac{\text{True positives}}{\text{True positives + False negatives}}$$

- F1 score: Measures the balance between precision and recall. It is computed as follows:

$$2 * \left(\frac{\text{Precision * Recall}}{\text{Precision + Recall}} \right)$$

- Confusion Matrix: Visualization showing correct and incorrect predictions in matrix form, specifically, true positives, true negatives, false positives, and false negatives.

To measure the impact of the backbone on the results, another version of the model was trained using *mobilenetv3_small_100.lamb_in1k*[5] pre-trained on *ImageNet-1k*[22] at the feature extractor, instead of *swinv2_tiny_window8_256*[4].

The main reason for choosing *mobilenetv3_small_100.lamb_in1k*[5] as an alternative is that, since this backbone was designed to be used on phones, in the event of obtaining results similar to those observed with *swinv2_tiny_window8_256*[4] the technical requirements for using the model would be significantly reduced, and it would also allow the first step of the feature extractor to be performed as frames are captured, thereby reducing the time required for each inference.

Additionally, since this backbone is also present in *timm*[23], implementing both backbones requires few changes to the code.

The code used, along with an exported version of the model with the SwinV2 backbone in ONNX format, can be found at the following GitHub:

https://github.com/ocasalsmo/Edicio_de_video_multicamera

4. Results

At the end of the training process, a model capable of recommending the most appropriate viewpoint in a multi-camera environment was obtained.

Training the model using the *swinv2_tiny_window8_256*[4] backbone took 1 hour and 49 minutes, whereas training with *mobilenetv3_small_100.lamb_in1k*[5] required 1 hour and 21 minutes.

As shown in Figure 7, both training times are comparable to those reported in *Improving Multi-Camera View Recommendation with Temporal and Camera Embeddings*[3] and are substantially shorter than those reported for the TC-Transformer.

Model	Training time
TC Transformer	28 hours
Model with temporal and camera *embeddings* (Paper *Improving Multi-Camera View Recommendation with Temporal and Camera Embeddings*[3])	2 hours 5 minutes
Model with temporal and camera *embeddings* (*mobilenetv3_small_100.lamb_in1*[5])	**1 hour 21 minutes**
Model with temporal and camera *embeddings* (*swinv2_tiny_window8_256*[4])	**1 hour 49 minutes**

Figure 7: *Training time of each model, for the paper's models, data was taken from Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3]*, where the two models were run on a system with a GeForce RTX 2080 Ti and a batch size of 2.*

The difference in training time between our model using the SwinV2 backbone and the model presented in *Improving Multi-Camera View Recommendation with Temporal and Camera Embedding*[3] can be attributed to two main factors. First, this work employs the tiny variant of the backbone, which is considerably more lightweight. Second, a larger batch size was used during training.

To evaluate the effectiveness of the training process, two metrics were considered: loss and accuracy.

As shown in figures 8 and 9, the training and validation sets metrics follow similar trends throughout all epochs. This suggests that neither model exhibits signs of overfitting.

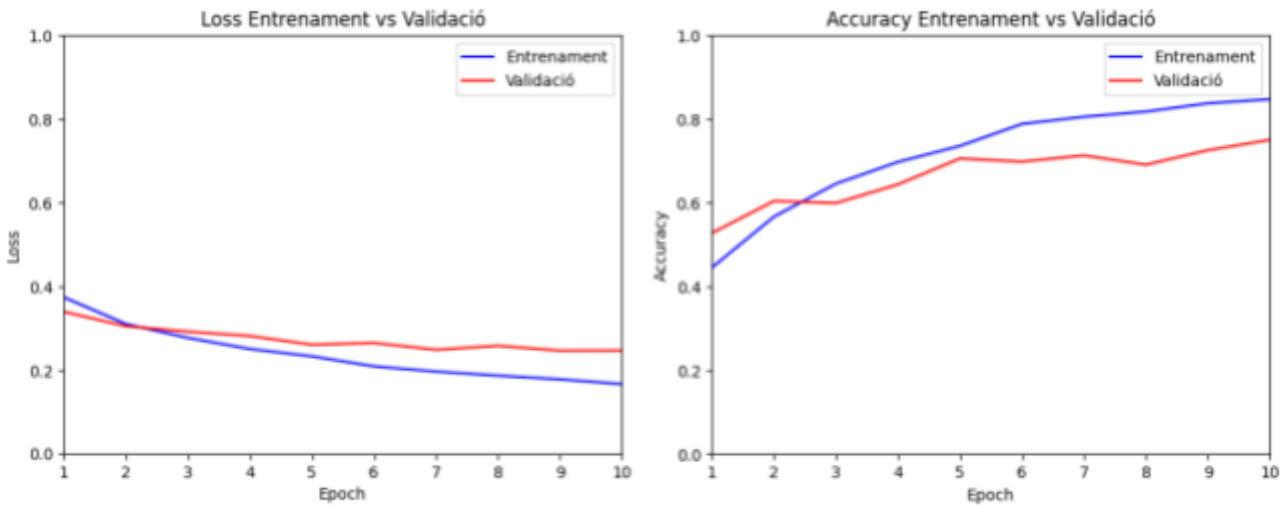


Figure 8: *Loss and accuracy on the training and validation set for each epoch in the model with Swinv2 as the backbone.*

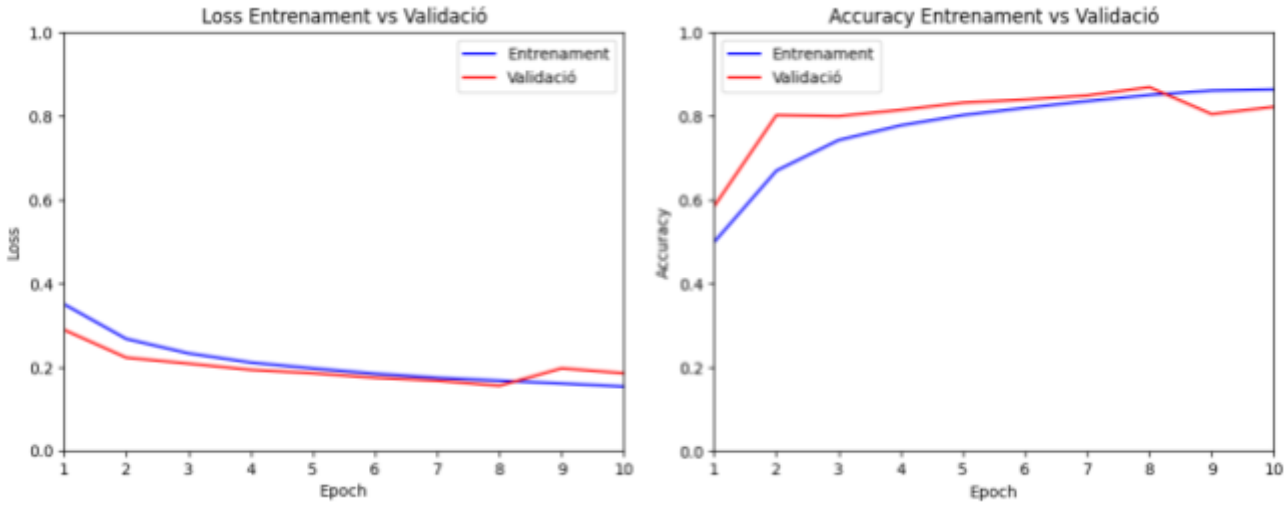


Figure 9: *Loss and accuracy on the training and validation set for each epoch in the model with MobileNetV3 as the backbone.*

Figure 10 shows the different quality metrics obtained by using the model on the test set.

Metric	Value (SwinV2)	Value (MobileNetV3)
Accuracy	71.21%	43'05%
Loss	0.28	0.49
Precision	68.75%	41.63%
Recall	69.41%	38.43%
F1 score	68.45%	36.35%

Figure 10: Quality metrics for each backbone.

The model using the SwinV2 backbone achieved a high accuracy, indicating that the majority of camera selections were predicted correctly.

In contrast, the model using the MobileNetV3 backbone achieved a substantially lower accuracy. This difference suggests that the choice of backbone has an impact on the model's ability to correctly identify the appropriate viewpoint.

The model using the SwinV2 backbone achieved a precision close to 70%, indicating that most of its predicted camera selections are reliable.

By contrast, the precision obtained with the MobileNetV3 backbone is considerably lower, demonstrating that this backbone negatively affects the reliability of the model's predictions.

As shown in Figure 11, the precision achieved by both models exceeds that reported for the TC-Transformer in the original paper.

Model	Precision(%)
TC-Transformer(ViVi)	21.29
TC-Transformer(Swin)	22.48
Model with temporal and camera *embeddings* (MobileNetV3)	41.63
Model with temporal and camera *embeddings* (SwinV2)	68.75

Figure 11: Precision of the obtained model and *the one shown in Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[19] *for all the tested backbones.*

The average recall achieved by the model using the SwinV2 backbone is close to 70%, a value comparable to its precision. This indicates that the model is able to identify the correct viewpoint in the majority of cases.

Consistent with the previous metrics, the model using the MobileNetV3 backbone achieved a substantially lower recall, further confirming the negative impact of replacing the backbone.

Finally, the F1-score obtained by the model using the SwinV2 backbone is also close to 70%, in line with the corresponding precision and recall values. This result indicates that the model achieves a good balance between the reliability of its predictions and its ability to identify the most appropriate viewpoint.

As with the other evaluation metrics, the model using the MobileNetV3 backbone achieved a lower F1-score, further demonstrating the negative impact of the backbone replacement on the model's overall performance.

To investigate whether the model exhibited a higher error rate for specific cameras, a confusion matrix was generated using the predictions of the model with the SwinV2 backbone.

As shown in Figure 12, although the majority of predictions are correct for all cameras, Cameras 4 and 5 exhibit the highest number of misclassifications. Most of these errors occur because the model incorrectly selects frames from Camera 2 instead of those captured by Cameras 4 and 5.

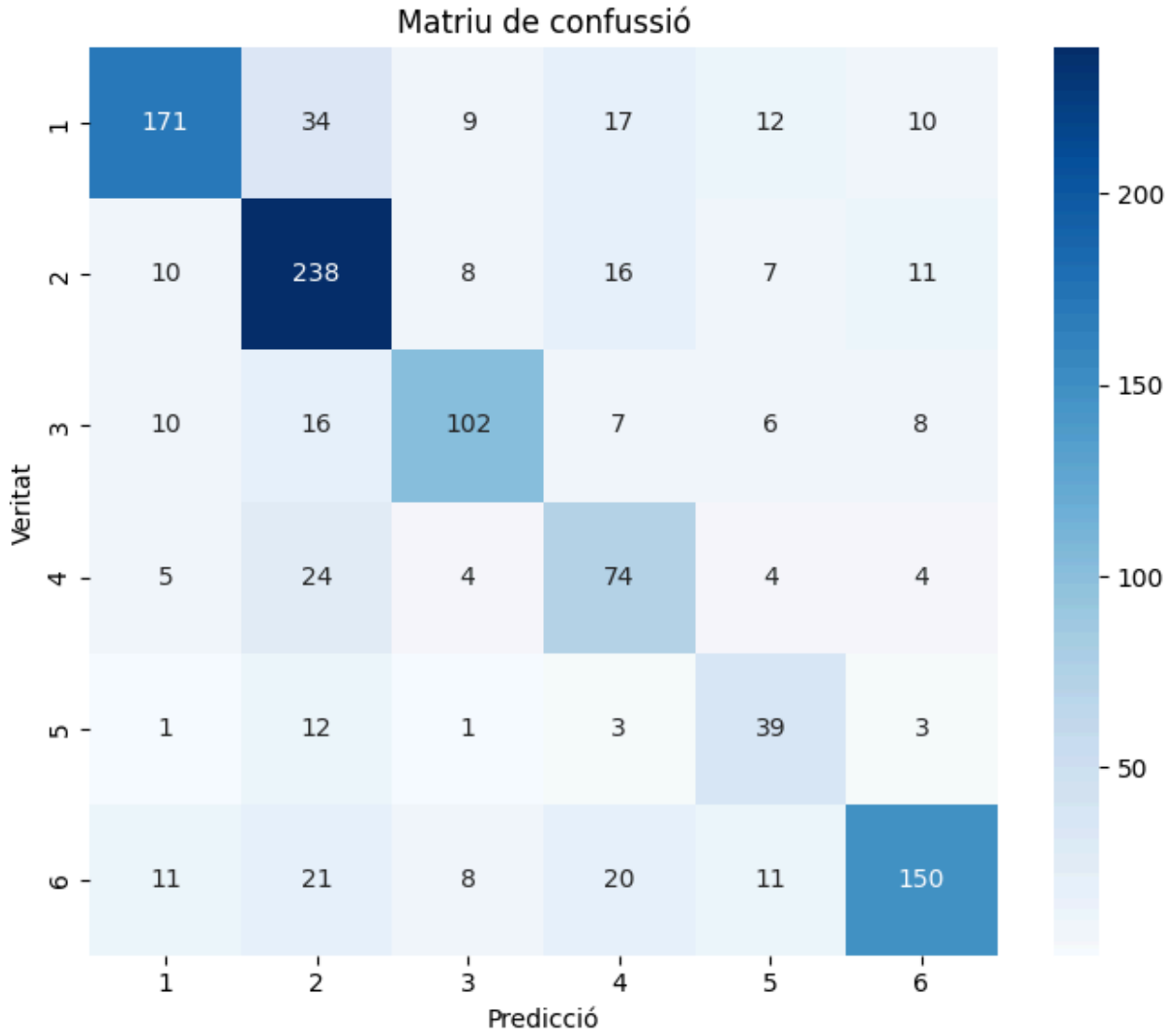


Figure 12: *Confusion matrix with SwinV2 backbone*

Finally, to complement the information provided by the confusion matrix, Figure 13 reports the precision, recall, and F1-score obtained for each camera individually on the test set.

As shown in Figure 13, and consistent with the observations from the confusion matrix, Cameras 4 and 5 achieved lower precision than the remaining cameras.

Camera	Precision(%)	Recall (%)	F1-score (%)
CAM1	82.21	67.59	74.19
CAM2	68.98	82.07	74.96
CAM3	77.27	68.46	72.60
CAM4	54.01	64.35	58.73
CAM5	49.37	66.10	56.52
CAM6	80.65	67.87	73.71

Figure 13: *Precision, Recall and F1-score of each camera with backbone SwinV2.*

By examining several randomly selected moments from Cameras 2, 4, and 5 across the two videos in the test set, it can be observed that the corresponding viewpoints are often highly similar. This suggests that the model has difficulty distinguishing the most appropriate camera when the candidate viewpoints are visually alike.

Figure 14 shows an example of a misclassified prediction, where the model selected Camera 2 instead of Camera 5. As can be seen, the two frames are visually very similar, which helps explain the model's incorrect prediction.

Figure 14: *Cameras 2 and 4 of video 0000 in the test set.*

A confusion matrix was also generated for the model using the MobileNetV3 backbone to determine whether its performance was particularly affected by any specific camera.

As shown in Figure 15, the model correctly classified the majority of samples corresponding to Cameras 1 and 2. However, starting from Camera 3, the number of correct predictions decreases substantially compared with the model using the SwinV2 backbone.

The most notable observation from the confusion matrix is that, for Cameras 4 and 5, which already posed difficulties for the model using the more powerful SwinV2 backbone, the MobileNetV3-based model produces more incorrect predictions than correct ones. Furthermore, in most cases where Cameras 4 or 5 correspond to the ground-truth viewpoint, the model selects Camera 2 instead.

As discussed previously, Camera 2 often provides viewpoints that are visually similar to those of Cameras 4 and 5. Therefore, the difficulty of distinguishing between highly similar viewpoints is even more pronounced when using the MobileNetV3 backbone.

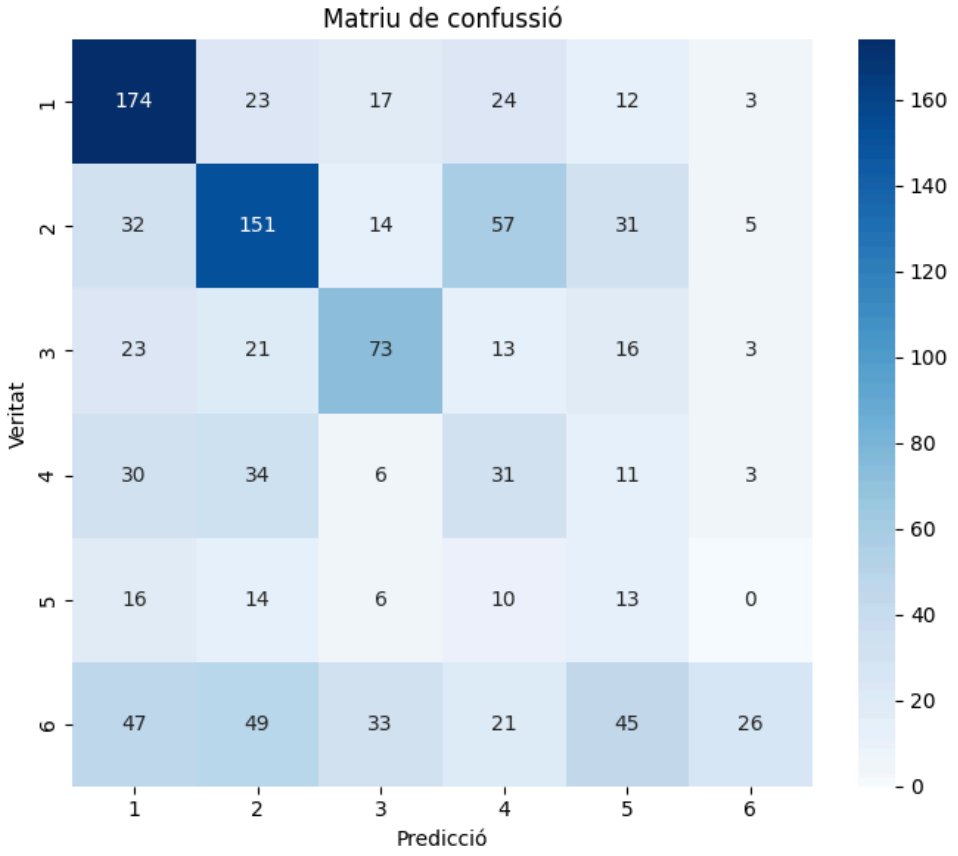


Figure 15: *Confusion matrix of the model with the MobileNetV3 backbone*

5. Conclusions and future directions

In this work, the architecture proposed in *Improving Multi-Camera View Recommendation with Temporal and Camera Embeddings*[3] was successfully replicated, resulting in a model capable of selecting the most appropriate camera to use at a given moment in a multi-camera environment.

The obtained results are consistent with those reported in the aforementioned paper. The main difference lies in the shorter training time achieved in this work, which can be attributed to the larger batch size and the use of tiny variants of backbones.

The accuracy and precision achieved by the proposed model indicate that the majority of camera selections are predicted correctly. However, the model encounters difficulties when multiple cameras provide highly similar viewpoints.

As a result, the model is less suitable for scenarios in which the visual differences between candidate viewpoints are minimal.

On the other hand, the precision achieved by the proposed model is higher than that reported for the model presented in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[19], while also requiring substantially less training time.

Additionally, an alternative version of the model was trained using *mobilenetv3_small_100.lamb_in1*[5] as the feature extraction backbone instead of *swinv2_tiny_window8_256*[4]. The objective was to evaluate whether the favorable results obtained with the original architecture could be maintained using a more computationally efficient backbone.

Although the training time was reduced, the accuracy, precision, recall, and F1-score demonstrate that replacing the backbone with a weaker one has a negative impact on the model's performance.

Moreover, the difficulties already observed with the more powerful backbone when distinguishing between highly similar candidate frames become even more pronounced in this version of the model.

In conclusion, this work has produced a model that achieves higher precision than the one presented in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows*[19], while requiring significantly less training time and fewer computational resources. Among the two evaluated backbones, the version using *swinv2_tiny_window8_256*[4] achieved the best overall performance, highlighting the importance of the feature extraction backbone within this architecture.

Regarding the project planning, the initial implementation strategy consisted of extracting the frames from each TAR archive on demand, rather than extracting the entire dataset beforehand.

However, this approach introduced a significant data-loading bottleneck during training. Combined with the limited experience of the author in using PyTorch, this issue ultimately led to the decision to forgo the planned quantization of the feature extraction backbone.

From a sustainability perspective, eliminating the temporal and contextual transformers employed by previous approaches reduces both the computational resources and the training time required by the model.

The social impact of the proposed system is also limited. Since the model is only capable of selecting the most appropriate viewpoint when a camera switch is to be performed, it is not intended to replace professional video editors but rather to assist them by streamlining the editing process.

As a result, editors can focus on determining the most appropriate moments to transition between cameras instead of spending time deciding which viewpoint should be displayed.

Furthermore, the model's precision is still far from perfect, and it struggles when the candidate viewpoints are highly similar. Consequently, human supervision remains necessary to review the model's recommendations and intervene whenever the selected frame is not the most appropriate one.

From an ethical perspective, the proposed system neither promotes nor discourages any particular ethical behavior. The developed model simply recommends the most appropriate viewpoint at each moment in a multi-camera environment. Therefore, the ethical implications of its use depend entirely on the user.

Finally, regarding future work, one possible research direction is to evaluate the proposed model on multi-camera videos belonging to domains that are not represented in the TVMCE dataset, following the approach proposed in *Pseudo Dataset Generation for Out-of-Domain Multi-Camera View Recommendation*[2], in order to assess its generalization capabilities.

Building upon the ideas presented in that work, another promising direction would be to fine-tune the proposed model using simulated multi-camera environments in order to specialize it for a specific application domain, such as film production.

This idea is based on the following observations:

- Many shot transitions in a video correspond to changes between camera viewpoints.

- In a multi-camera environment, cameras are typically stationary and capture the same scene from different viewpoints and at different scales, ranging from wide shots to close-up views of the action

Consequently, it is possible to simulate a multi-camera environment from a conventional video by treating each distinct viewpoint as if it had been captured by a different camera and defining the candidate for each moment according to their visual similarity.

To implement this idea, a collection of public-domain films, such as those available through WikiFlix, could be used to build a domain-specific dataset. After collecting the videos, each film would be divided into individual frames.

Once the frames have been extracted, a clustering algorithm could be applied independently to each film in order to group visually similar frames and assign an identifier to every cluster. Each cluster would then represent a pseudo-camera within the simulated multi-camera environment.

After constructing the simulated environment, candidate frames for each timestamp could be generated by applying cosine similarity to all available frames and selecting, from each pseudo-camera, the frame that is most similar to the one used at that particular moment in the original film.

The resulting dataset could then be used to fine-tune the model proposed in this work, allowing it to specialize in recommending the most appropriate viewpoint for film production.

The main limitation of this approach is the difficulty of evaluating the resulting model. A reliable evaluation would require access to the original recordings from each physical camera used during the production of at least one real multi-camera film. Since the simulated candidate viewpoints are not derived from an actual editing process, they can not be used for testing as they may introduce biases that could make the model appear either more or less effective than it truly is.

In addition to specializing the proposed model, another possible research direction would be to evaluate the impact of replacing the tiny backbone with a larger variant, analyzing how this change affects both the training time and the model's performance.

The experiments conducted in this work have shown that replacing a powerful backbone such as *swinv2_tiny_window8_256*[4] with a lighter alternative degrades the model's performance. Therefore, it would be worthwhile to investigate whether employing a more powerful backbone could further improve the accuracy of the model's viewpoint recommendations.

Another possible improvement would be to incorporate audio into the feature vector. The additional audio cues could help the model identify the most relevant events occurring in each scene more effectively, leading to better-informed viewpoint recommendations.

The main challenge associated with this approach is the limited availability of multi-camera datasets that provide synchronized audio together with multiple viewpoints.

Another possible improvement would be to incorporate audio information into the feature vector. The additional audio cues could help the model identify the most relevant events occurring in each scene more effectively, leading to better-informed viewpoint recommendations.

The main challenge associated with this approach is the limited availability of multi-camera datasets that provide synchronized audio together with multiple viewpoints.

The integration of both visual and auditory information has already proven effective in other computer vision tasks, particularly in event recognition. An example is the approach presented in *Multimodal Attentive Fusion Network for audio-visual event recognition*[25].

Given the role of audio in the aforementioned work, incorporating audio into the proposed model could provide additional information about the events taking place within a scene, enabling the model to make more informed viewpoint selection decisions.

Another interesting research direction would be to evaluate how the proposed architecture performs when applied to a single wide-angle video instead of a conventional multi-camera setup. In this scenario, the wide-angle frame would be divided into multiple smaller windows, following an approach similar to that proposed in *Real Time GAZED: Online Shot Selection and Editing of Virtual Cameras From Wide-Angle Monocular vídeo Recording*[8] .

To implement this idea, the wide-angle frame could be partitioned into multiple smaller regions, each assigned a unique identifier. These regions would then act as virtual cameras, effectively simulating a multi-camera environment that could be processed by the model developed in this work.

This approach would make it possible to investigate whether the proposed architecture is applicable not only to traditional multi-camera productions but also to videos generated by extracting multiple virtual shots from a single static wide-angle recording, as is often done in low-budget productions.

Finally, the proposed model could be combined with another model designed to determine when a shot transition should occur, rather than which viewpoint should be selected. Such an approach could be integrated with methods such as the one presented in *When and How to Cut Classical Concerts? A Multimodal Automated vídeo Editing Approach*[16] .

6. Glossary

- **Multicamera**: A filming method in which multiple cameras record the same scene from different viewpoints, and then the best angles are used to create a single video.
- **Self-attention**: A mechanism widely used in machine learning models to evaluate the relevance of each element in a data sequence with respect to the others, enabling the model to capture contextual relationships.
- **Transformer:** A deep learning architecture that models the relationships between elements in a data sequence in order to learn contextual information.

- **Embedding:** A technique widely used in machine learning to represent complex inputs, such as words, as numerical vectors that can be processed by a model.

- **MLP**: A feedforward neural network composed of fully connected layers with nonlinear activation functions. MLPs are widely used because of their ability to learn complex nonlinear relationships between input data.

- **TVMCE dataset:** The dataset used in this work to train and evaluate the proposed model. It was first introduced in *Temporal and Contextual Transformer for Multi-Camera Editing of TV Show*[19]

- **Backbone:** Feature extractor used in Deep Learning models to extract features from images.

7. Bibliography

1. Rao, A., Jiang, X., Wang, S., Guo, Y., Liu, Z., Dai, B., Pang, L., Wu, X., Lin, D., & Jin, L. (2022). *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows* (arXiv:2210.08737). arXiv. https://doi.org/10.48550/arXiv.2210.08737

2. Lee, K.-Y., Zhou, Q., & Nahrstedt, K. (2024). *Pseudo Dataset Generation for Out-of-Domain Multi-Camera View Recommendation* (arXiv:2410.13585). arXiv. https://doi.org/10.48550/arXiv.2410.13585

3. Lee, K.-Y., Zhou, Q., & Nahrstedt, K. (2025). Improving Multi-Camera View Recommendation with Temporal and Camera Embedding. *2025 21st International Conference on Intelligent Environments (IE)*, 1-5. https://doi.org/10.1109/IE64880.2025.11130132

4. Liu, Z., Hu, H., Lin, Y., Yao, Z., Xie, Z., Wei, Y., Ning, J., Cao, Y., Zhang, Z., Dong, L., Wei, F., & Guo, B. (2021, novembre 18). *Swin Transformer V2: Scaling Up Capacity and Resolution*. arXiv.Org. https://arxiv.org/abs/2111.09883v2

5. Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., & Adam, H. (2019). *Searching for MobileNetV3* (arXiv:1905.02244). arXiv. https://doi.org/10.48550/arXiv.1905.02244

6. Chouksey, A., Rajan, A. K., Gurjar, V., Tiwari, R., & Mishra, P. K. (2026). The green paradox: The climate, environmental, and sustainability implications of artificial intelligence. *Global Environmental Change Advances*, *6*, 100029. https://doi.org/10.1016/j.gecadv.2025.100029

7. Reisz, K. (s.d.). *The technique of film editing*.

8. Achary, S., Girmaji, R., Deshmukh, A. A., & Gandhi, V. (2024). Real Time GAZED: Online Shot Selection and Editing of Virtual Cameras from Wide-Angle Monocular Video Recordings. *2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 4096-4104. https://doi.org/10.1109/WACV57701.2024.00406

9. He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition* (arXiv:1512.03385). arXiv. https://doi.org/10.48550/arXiv.1512.03385

10. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248-255. https://doi.org/10.1109/CVPR.2009.5206848

11. Bai, J., Xia, M., Fu, X., Wang, X., Mu, L., Cao, J., Liu, Z., Hu, H., Bai, X., Wan, P., & Zhang, D. (2025). *ReCamMaster: Camera-Controlled Generative Rendering from A Single Video* (arXiv:2503.11647). arXiv. https://doi.org/10.48550/arXiv.2503.11647

12. Sobchyshak, O., Berrezueta-Guzman, S., & Wagner, S. (2026). Pushing the boundaries of immersion and storytelling: A technical review of Unreal Engine. *Displays*, *91*, 103268. https://doi.org/10.1016/j.displa.2025.103268

13. Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lučić, M., & Schmid, C. (2021). *ViViT: A Video Vision Transformer* (arXiv:2103.15691). arXiv. https://doi.org/10.48550/arXiv.2103.15691

14. Liu, Z., Ning, J., Cao, Y., Wei, Y., Zhang, Z., Lin, S., & Hu, H. (2021). *Video Swin Transformer* (arXiv:2106.13230). arXiv. https://doi.org/10.48550/arXiv.2106.13230

15. Kay, W., Carreira, J., Simonyan, K., Zhang, B., Hillier, C., Vijayanarasimhan, S., Viola, F., Green, T., Back, T., Natsev, P., Suleyman, M., & Zisserman, A. (2017). *The Kinetics Human Action Video Dataset* (arXiv:1705.06950). arXiv. https://doi.org/10.48550/arXiv.1705.06950

16. Gonzálbez-Biosca, D., Cabacas-Maso, J., Ventura, C., & Benito-Altamirano, I. (2025). When and How to Cut Classical Concerts? A Multimodal Automated Video Editing Approach. *Proceedings of the 3rd International Workshop on Multimedia Content Generation and Evaluation: New Methods and Practice, McGE '25*, 98-106. https://doi.org/10.1145/3746278.3759387

17. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). *Attention Is All You Need* (arXiv:1706.03762). arXiv. https://doi.org/10.48550/arXiv.1706.03762

18. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows* (arXiv:2103.14030). arXiv. https://doi.org/10.48550/arXiv.2103.14030

19. Rao, A., Jiang, X., Wang, S., Guo, Y., Liu, Z., Dai, B., Pang, L., Wu, X., Lin, D., & Jin, L. (2022). *Temporal and Contextual Transformer for Multi-Camera Editing of TV Shows* (arXiv:2210.08737). arXiv. https://doi.org/10.48550/arXiv.2210.08737

20. *Pie Chart Creator*. (s.d.). Recuperat 17 de maig de 2026, de https://geographyfieldwork.com/PieChartCreator.html

21. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., & Gimelshein, N. (s.d.). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*.

22. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., & Fei-Fei, L. (2014, setembre 1). *ImageNet Large Scale Visual Recognition Challenge*. arXiv.Org. https://arxiv.org/abs/1409.0575v3

23. Wightman, R., Raw, N., Soare, A., Arora, A., Ha, C., Reich, C., Guan, F., Kaczmarzyk, J., MrT23, Mike, SeeFun, Contrastive, Rizin, M., Hyeongchan Kim, Kertész, C., Dushyant Mehta, Cucurull, G., Kushajveer Singh, Hankyul, … Uchida, Y. (2023). *rwightman/pytorch-image-models: V0.8.10dev0 Release* (Versió v0.8.10dev0) [Programari]. Zenodo. https://doi.org/10.5281/ZENODO.4414861

24. Loshchilov, I., & Hutter, F. (2019). *Decoupled Weight Decay Regularization* (arXiv:1711.05101). arXiv. https://doi.org/10.48550/arXiv.1711.05101

25. Brousmiche, M., Rouat, J., & Dupont, S. (2022). Multimodal Attentive Fusion Network for audio-visual event recognition. *Information Fusion*, *85*, 52-59. https://doi.org/10.1016/j.inffus.2022.03.001